

Documentation de réponse au cahier des charges SAE 3.02



1. Éléments implémentés

L'ensemble des objectifs définis dans le cahier des charges a été implémenté et validé fonctionnellement. Le projet répond ainsi pleinement aux exigences techniques et pédagogiques attendues.

1.1 Architecture distribuée client–serveur

Une architecture distribuée complète a été conçue et mise en œuvre. Elle est composée :

- d'un serveur maître (Master),
- de plusieurs routeurs virtuels interconnectés,
- de plusieurs clients capables d'échanger des messages de manière anonyme.

L'architecture a été testée sur plusieurs machines virtuelles, conformément aux contraintes imposées dans le cahier des charges.

1.2 Communication réseau

La communication entre les différents composants repose sur l'utilisation des sockets Python.

Les échanges sont assurés entre :

- les clients et le premier routeur de la chaîne,
- les routeurs entre eux,
- le dernier routeur et le client destinataire.

Le routage multi-sauts est entièrement fonctionnel et repose sur un chaînage dynamique des routeurs.

La gestion de plusieurs connexions simultanées est assurée grâce à l'utilisation de threads, permettant la mise en œuvre d'un serveur multi-clients robuste.

1.3 Routage en oignon (Onion Routing)

Le principe du routage en oignon a été intégralement implémenté, en s'inspirant du fonctionnement de Tor.

Chaque message est encapsulé dans plusieurs couches de chiffrement successives, chaque couche étant destinée à un routeur spécifique.

Chaque routeur :

- ne connaît que son voisin précédent et son voisin suivant,
- déchiffre uniquement la couche qui lui est destinée,
- transmet le reste du message au prochain saut.

Ce mécanisme garantit l'anonymisation des communications sur l'ensemble du réseau distribué.

1.4 Chiffrement asymétrique

Un algorithme de chiffrement asymétrique de type RSA simplifié a été implémenté manuellement, sans recours à une bibliothèque cryptographique externe, conformément aux contraintes du projet.

Chaque routeur dispose :

- d'une clé publique, transmise aux clients par le Master,
- d'une clé privée, stockée localement.

Les clients chiffrent les messages en plusieurs couches successives, chaque couche étant chiffrée avec la clé publique du routeur correspondant au chemin choisi.

1.5 Base de données MariaDB

Une base de données MariaDB est utilisée afin de stocker les informations nécessaires au fonctionnement du système, notamment :

- les clés de chiffrement,
- les informations de routage,
- les données utiles au pilotage du réseau.

Les accès à la base sont réalisés via des requêtes SQL intégrées au code Python.

1.6 Interfaces graphiques

Des interfaces graphiques ont été développées à l'aide de Qt :

- une interface client permettant la connexion au réseau et l'envoi de messages de manière non bloquante,
- une interface Master permettant la visualisation de la topologie du réseau, la distribution des clés et l'affichage des journaux.

Ces interfaces facilitent l'utilisation du système et améliorent la compréhension du fonctionnement global de l'architecture.

2. Éléments non implémentés

Aucun élément obligatoire du cahier des charges n'a été laissé non implémenté.

Toutes les fonctionnalités demandées dans le sujet ont été réalisées et intégrées dans le projet final.

3. Conformité au cahier des charges

Le projet respecte l'ensemble des contraintes imposées :

- langage de programmation : Python,
- utilisation exclusive des bibliothèques autorisées
- implémentation manuelle des mécanismes de chiffrement,

- structure de code claire, modulaire et commentée,
- dépôt Git conforme avec un historique de développement exploitable.

Algorithme de chiffrement

Forces et faiblesses

1. Principe général

Le projet utilise un chiffrement asymétrique de type RSA simplifié, implémenté manuellement en Python.

Chaque routeur possède une paire de clés :

- une clé publique, transmise aux clients,
- une clé privée, conservée localement.

Les clients chiffrent les messages en plusieurs couches successives, chaque couche étant chiffrée avec la clé publique d'un routeur différent.

Chaque routeur ne peut déchiffrer que la couche qui lui est destinée, ce qui respecte le principe du routage en oignon.

2. Forces du chiffrement

2.1 Respect du fonctionnement du RSA

L'algorithme respecte les principes de base du chiffrement RSA :

- génération de clés publique et privée,
- chiffrement avec la clé publique,
- déchiffrement avec la clé privée.

Cela permet de bien comprendre le fonctionnement du chiffrement asymétrique.

2.2 Implémentation conforme au cahier des charges

Le chiffrement est entièrement développé à la main, sans utiliser de bibliothèque cryptographique externe.

Ce choix respecte les contraintes du projet et met en valeur la compréhension des mécanismes internes du chiffrement.

2.3 Bonne intégration au routage en oignon

Le chiffrement par couches successives est parfaitement adapté au routage en oignon :

- chaque routeur ne déchiffre qu'une seule couche,
- aucun routeur ne connaît le message complet ni le chemin total.

Cela garantit une anonymisation fonctionnelle des échanges.

2.4 Adapté à un cadre pédagogique

Dans un environnement de test et de démonstration, l'algorithme est fiable et simple à comprendre.

3. Faiblesses et limites

3.1 Clés de petite taille

Les nombres premiers utilisés pour générer les clés sont de petite taille.

Cela rend le chiffrement insuffisant face à des attaques réelles, mais suffisant dans le cadre de notre projet

Bilan:

Au final, si on se réfère au premier dossier de rendu du projet et à notre planning, notre projet a été mené de bout en bout avec toutes les étapes et le plan technique bien réalisé. Nous sommes très satisfait du travail produit et nous remercions le professeur pour nous avoir permis de réaliser ce projet.